



Quadratic in Theory, Flat in Hardware: Evaluating Grover Search on Small Games Using Qiskit and IBM Quantum

H. Botezatu, R. Kapoor, L. Milieška, F. van Onna, D. Lenta, R. van Unen

TU Delft, Quantum Minor Programme, Delft, The Netherlands

Abstract

This report evaluates whether student-accessible quantum hardware is currently capable of running Grover's search algorithm on game-like problems. Grover provides a quadratic speed-up for unstructured search, but its practical value depends on oracle depth and hardware noise. Three cases are studied: Lights Out, small Sudoku, and procedural map generation via Wave Function Collapse (WFC), each with a classical baseline and a Grover-based formulation implemented in Qiskit and, where feasible, executed on IBM hardware. In noise-free simulation, Grover amplification is observed as expected. On hardware, performance degrades strongly with circuit depth: Lights Out (2×2) drops from $\approx 96.13\%$ (ideal) to $\approx 16.42\%$ (IBM), and WFC (3×3) from 99.6% valid to $\approx 1.4\%$ valid. Overall, classical structure-aware solvers remain superior at the problem sizes currently runnable.

Introduction

Quantum computing offers new approaches to computational problems that are difficult to solve efficiently using classical computers. One of the most well-known quantum algorithms is **Grover's search algorithm**, which provides a quadratic speed-up for searching an unsorted database [1]. Given a search space of size N , a classical algorithm requires $O(N)$ checks in the worst case, whereas Grover's algorithm can find a marked solution using only $O(\sqrt{N})$. This quadratic improvement is proven to be optimal for unstructured search problems.

Despite this theoretical advantage, a quantum speed-up does not automatically translate into practical usefulness. If the size of the search space grows exponentially with the input size, a square-root reduction still results in exponential complexity. Consequently, for many NP-hard problems, Grover's algorithm merely reduces the exponent by a factor of two. This raises important questions regarding the feasibility of quantum approaches on near-term quantum hardware, especially for small problem instances and platforms that are accessible to students and researchers, such as cloud-based quantum computers and simulators.

For this reason, it is essential to study concrete and well-defined problems in order to assess both the potential and the limitations of Grover's algorithm in practice. In particular, games and puzzle-like problems form an attractive testbed for quantum algorithms: they are intuitive, naturally expressed as search problems, and allow for direct comparison with efficient classical solutions. The main objective of this project is to investigate whether currently available, student-accessible quantum computing technology is sufficiently mature to meaningfully implement Grover's algorithm for such problems, and to determine at what scale and under which constraints this is possible.

To this end, Grover's algorithm is applied to three representative game-related problems, and the quantum approach is compared to classical solution methods:

- **Lights Out:** Lights Out is a puzzle played on a grid (traditionally 5×5 , though other sizes are also considered) in which pressing a cell toggles the state of that cell and its orthogonally adjacent neighbors. Starting from a random configuration, the objective is to turn off all lights. While this problem can be solved efficiently using classical linear algebra tech-

niques, it can also be formulated as a search problem. The use of Grover’s algorithm is investigated to determine whether any advantage arises compared to the classical solution.

- **Sudoku (small instances):** Small Sudoku instances that are compatible with current quantum hardware are studied. A 2×2 Sudoku is implemented using Grover’s algorithm and executed on real IBM quantum hardware, while a 4×4 Sudoku is implemented and evaluated using Qiskit simulations. The quantum approach is compared against a classical solver, and differences in runtime complexity and resource requirements are analyzed.
- **Procedural Map Generation:** Wave Function Collapse (WFC) is a classical algorithm used for generating tile-based maps under local adjacency constraints, commonly applied in game development. A quantum variant of this approach is implemented by encoding the adjacency constraints into a quantum oracle and applying Grover’s algorithm to search for valid tile configurations. The quantum method is compared to its classical counterpart, which typically scales with the size of the generated map.

For each problem, both the theoretical runtime complexity and the practical performance are analyzed. Theoretical analysis is based on the size of the search space and the expected behavior of Grover’s algorithm, while practical performance is evaluated using simulation results and executions on real quantum hardware. Additional practical considerations, such as the required number of qubits, circuit depth, number of ancilla qubits, and overall execution time, are also taken into account. Together, these analyses provide insight into the current capabilities and limitations of applying Grover’s algorithm to game-like problems using student-accessible quantum computing platforms. The code used during the project can be found in Appendix B.

Methodology

A common methodology is used across all three problems: Lights Out, Sudoku, and Wave Function Collapse (WFC), to ensure fair and consistent comparison. Each problem is formulated as a Boolean search problem, where candidate solutions are mapped to computational basis states of a quantum register. A reversible oracle U_f is constructed to evaluate the problem constraints and mark valid solutions by applying a phase flip.

Figure 1 shows the general structure of a Grover iteration used throughout this work. The data qubits are initialized in the $|0\rangle$ state and transformed into a uniform superposition using Hadamard gates. An ancilla qubit, initialized in the $|1\rangle$ state and placed into superposition, enables phase kickback. The oracle U_f marks valid solutions, after which the diffusion operator D performs an inversion about the mean, amplifying the probability of measuring marked states.

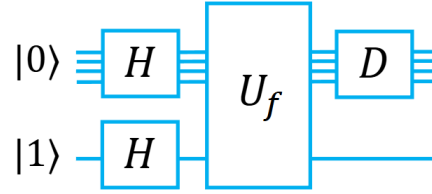


Figure 1: General circuit structure of Grover’s algorithm, consisting of state preparation, oracle evaluation, and diffusion [2].

Oracle construction is achieved by translating classical constraint checks into reversible quantum logic. Boolean conditions are implemented using combinations of CNOT and multi-controlled gates, allowing the oracle to evaluate whether a candidate solution satisfies all required constraints. Ancilla qubits are used to store intermediate results during constraint evaluation and are uncomputed where possible to preserve reversibility and avoid residual entanglement.

The number of Grover iterations is chosen according to the theoretical optimum, approximately $\frac{\pi}{4}\sqrt{N/M}$, where N denotes the search space size and M the number of valid solutions. After the final iteration, the data qubits are measured in the computational basis.

All quantum circuits are implemented using Qiskit. Simulations are performed with Qiskit Aer, while selected small instances are executed on IBM quantum hardware. For each experiment, the number of qubits, transpiled circuit depth, gate counts, number of shots, success probability, and total runtime are recorded. These results form the basis for the performance analysis presented in the Results section.

Sudoku

Sudoku is a constraint satisfaction puzzle where each row, column and block must contain each symbol exactly once. This project focuses on small binary Sudoku variants that are able to run on a quantum computer: a 2×2 Sudoku (4 cells) and a 4×4 Sudoku (16 cells).

Classical approach: For baseline comparisons standard classical methods are used. The 2×2 case is solved by exhaustive checking of all assignments. For the 4×4 instances a backtracking solver is used with simple pruning (row/column/block constraints). These classical solvers exploit constraint structure and are extremely fast on classical computers for the instance sizes considered, typically finishing in milliseconds or less.

Quantum formulation: Each unknown cell is encoded as a qubit (one qubit per binary cell value) and build a reversible oracle U_f that marks assignments satisfying all row, column and block constraints. Ancilla qubits implement temporary parity and uniqueness checks using

CNOTs and multi-controlled gates, making the check reversible. The algorithm starts in an equal superposition over the $N = 2^{\text{possible values}}$ candidate assignments, and then repeats Grover iterations (oracle + diffusion). The approximate number of Grover iterations is

$$k \approx \frac{\pi}{4} \sqrt{\frac{N}{M}},$$

where M denotes the number of satisfying assignments. For a partially filled 4×4 puzzle with approximately six unknown variables, and using two qubits per unknown to encode each possible value, a total of 12 qubits is required. This results in a search space size of $N \approx 2^{12} = 4096$. Assuming a single valid solution ($M = 1$), the expected number of Grover iterations is on the order of 50.

A key practical point is that the main limiter for running these circuits on cloud quantum hardware is gate depth, not raw qubit count. Although providers such as IBM have devices with many physical qubits, deep circuits with many sequential two- (or more) qubit gates quickly accumulate errors and cause decoherence. To see more about the theoretical capabilities of a Quantum Computer, the sudoku algorithm was also run using the AerSimulator of Qiskit. However, classical simulation faces its own severe scalability constraints due to the exponential growth of quantum state space. Simulating a completely empty 4×4 Sudoku requires 32 logical qubits just to encode the variables. A statevector simulation of this size requires a minimum of 64 GB of RAM for the amplitudes alone, excluding overhead for ancilla qubits and gate operations. Furthermore, the search space of 2^{32} states would require over **50,000** Grover iterations, making full simulation unfeasible.

Therefore a partially filled 4×4 instance was chosen to keep the overall circuit depth and the number of multi-qubit gates low enough to obtain useful results. Also, optimizations in terms of the ancillae used had to be done. To minimize qubit overhead, the oracle implements a batch processing strategy where constraints are evaluated in blocks of four, instead of assigning a unique ancilla to every constraint, which would require much more qubits. For each block, constraints are computed in temporary ancillae, and their results are added to a results register using a multi-controlled X gate. The block ancillae are then uncomputed to prepare them for the next batch.

The total number of constraints proved to also be a critical balancing factor in the circuit design. Each additional constraint increases the computational cost in two ways: it requires more logical gates to verify (increasing the oracle depth) and often necessitates additional ancilla qubits or reuse cycles. Theoretically, a set of all the needed constraints is needed to ensure the puzzle has a unique solution ($M=1$), which allows Grover's algorithm to amplify the correct state probability to nearly 100%, if the number of iterations is sufficient. Therefore, a minimal set of 28 constraints was selected, to minimize qubit overhead, and careful classical pre-processing was done to get a unique

solution without adding redundant checks that would unnecessarily inflate the circuit depth.

Typical resource estimates for the partially filled 4×4 run on quantum hardware are:

- Data qubits: 12 (two per unknown cell as to encode 4 values).
- Ancilla qubits: ~ 13 –14
- Total qubits after overhead: ~ 25 –26. (depends on the placement and number of empty cells)
- Planned Grover iterations: $k \approx 50$.

Lights Out

Classical approach: The classical implementation is both accessible and efficient because the game structure of Lights Out lends itself to being translated into a linear algebra problem, by defining A as an $n^2 \times n^2$ matrix, where $A[i,j]$ is 1 if pressing button j toggles light i and 0 otherwise, and $\vec{b}$ as a vector of length n^2 , corresponding to the initial game state, where 1 means a light is on and 0 that it is off. Solving the game is then equivalent to solving $A\vec{x} = \vec{b}$, where $\vec{x}$ is a vector of length n^2 , indicating the lights that need to be toggled as 1 and the rest as 0. This can be done using Gaussian elimination in polynomial time ($O(n^6)$).

Quantum formulation: Each board cell is encoded as a single data qubit since Lights Out is a binary puzzle (0 = off, 1 = on). For an $n \times n$ board this requires n^2 data qubits. The oracle is constructed to recognize bitstrings that correspond to solutions of the linear system described above.

For each cell a temporary qubit is used to accumulate parity checks of the cell and its neighbors (right, down, left, up). These temporary parity qubits are computed with CNOTs from the relevant data qubits, then combined via a multi-controlled X (MCX) gate to flip the output qubit for valid configurations. After marking, the parity computation is uncomputed to restore ancillae to the initial state, making the oracle fully reversible.

A custom MCX synthesis[3, 4] is used which increases the number of ancilla qubits compared to the simplest decompositions, but it reduces the overall error rate. The oracle circuit therefore contains three logical stages: (1) constraint evaluation (parity computation), (2) phase-marking (multi-controlled phase flip), and (3) uncomputation of temporary qubits.

The algorithm begins by preparing the data qubits in an equal superposition with Hadamards. The output qubit is prepared in the $|-\rangle$ state. Grover iterations (oracle + difuser) are then applied. The code computes an estimate for the number of Grover iterations as

$$k \approx \frac{\pi}{4} \sqrt{\frac{2^{n^2}}{M}},$$

where M is the number of solutions (we use known sequence values[5] for solution counts) and n size of the game. For the 2×2 test instance ($n = 2$) used in our experiments the implementation calculates that $k = 3$.

Additionally, to reduce noise dynamical decoupling[6] was applied for the IBM hardware runs. It is intended to reduce decoherence and in practice is recommended when running long circuits.

Resource summary for the 2×2 runs reported here:

- Data qubits: $n^2 = 4$
- Temporary (parity) qubits: $n^2 = 4$
- Ancilla qubits for MCX synthesis: $n^2 - 2 = 2$
- Output/phase qubit: 1
- Total qubits: $3n^2 - 1 = 11$
- Logical gate count (untranspiled): 156
- Circuit depth (untranspiled): 63
- Estimated Grover iterations: 3

WFC

Wave Function Collapse (WFC) generates based on some given tile sets and constraints for the tile combination a map, which can be used in games such as Catan. When increasing the grid size to a very large size, the amount of grid combinations becomes exponentially large:

$$\text{DataSize} = |\mathcal{T}|^N$$

where $\mathcal{T}$ denotes the set of available tile types, $|\mathcal{T}|$ is the cardinality of this set (i.e., the number of distinct tile types), and N represents the total number of tiles in the grid. Consequently, DataSize corresponds to the total number of possible tile configurations. With the help of Grover's algorithm, the search for a valid grid configuration can be sped up quadratically.

Classical Approach: The classical baseline is implemented using a constraint-based Wave Function Collapse (WFC) algorithm. The grid consists of tiles that can take one of four possible states: Water, Beach, Grass, or Forest. The constraints to be satisfied is that each of those tile types can only border themselves and their direct neighbours in the previously listed order, so Water can only border itself and Beach, Beach can border itself, Water and Grass, et cetera. Each grid position is initialized with all tile options simultaneously, representing maximum uncertainty.

The algorithm proceeds iteratively by alternating between a collapse step and constraint propagation. During collapse, a grid position that still has multiple valid tile options is selected at random, and one option is chosen uniformly at random. This reduces the local uncertainty and introduces stochasticity into the generation process.

After collapsing a tile, adjacency constraints are enforced by propagating restrictions to neighboring grid positions. Tile options that are incompatible with neighboring states are removed. This propagation is repeated until no further changes occur, ensuring consistency across the grid.

These two steps are repeated until every grid position has exactly one remaining tile option, yielding a fully collapsed configuration that satisfies all local constraints. The resulting tile map serves as a classical reference for comparison with the quantum-based approach.

Quantum formulation: For the quantum approach, the same tile types are used as in the classical case (Water, Beach, Grass, and Forest). Since four tile types are required, each tile is encoded using two qubits. Consequently, the grid data alone requires

$$2NM,$$

where N denotes the number of rows and M the number of columns. Due to this large number of data qubits, constructing an efficient oracle requires minimizing the number of ancilla qubits while still correctly checking all adjacency constraints.

The oracle is constructed by evaluating local adjacency constraints between neighboring tiles and marking configurations that satisfy all constraints. Each constraint check compares the qubit encoding of two neighboring tiles and flips an ancilla qubit if an invalid combination is detected. These checks are implemented using multi-controlled X (MCX) gates acting on the data qubits that encode the tile values.

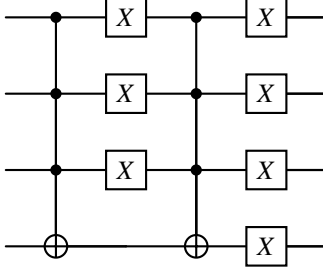
To limit the total number of ancilla qubits, adjacency checks are evaluated in batches. Within each batch, multiple constraint checks are performed in parallel, and their results are aggregated into a reserved ancilla qubit that represents whether all constraints in that batch are satisfied. The temporary ancilla qubits are then uncomputed and reused for subsequent batches. After all constraints have been evaluated, a multi-controlled Z gate is applied, conditioned on the constraint ancillae, to apply a phase flip to valid configurations. The constraint circuit is then inverted to restore all ancilla qubits to their initial states.

With this method for the 3×3 WFC instance are:

- Data qubits: 18 (2 per tile).
- Ancilla qubits: 8 (after batching and reuse strategy).
- Total qubits: 26.
- Grover iterations: 9 (estimated by sampling the initial probability of valid configurations and using it to approximate the optimal Grover rotation angle).

To simplify the implementation of the constraints and the resulting oracle, the tiles in the grid are encoded in two different ways: Either Water, Beach, Grass and Forest tiles

are encoded using the bitstrings '00', '01', '10', and '11' respectively, or in reverse order, depending on the location of the tile within the grid. We called these encodings A and B, which are used in the grid in a checkerboard pattern, such that tiles encoded using A only border tiles encoded using B. This simplifies the circuit needed to check a single constraint between two tiles to



Here, the top two qubits are the two qubits of the tile, the third qubit is the first qubit of one of the tiles neighbors and the last qubit is the ancilla in which the result of the constraint is stored. This circuit has been designed specifically for this ruleset, and needs to be checked both ways: if tile A with qubits A1 and A2 borders tile B with qubits B1 and B2, this circuit needs to be used once on A1, A2 and B1, and once on A1, B1 and B2, together producing two results stored in ancillae. This process then needs to be run for every two neighboring tiles, after which the results stored in the ancilla qubits are combined into a multi-controlled Z-gate. As mentioned previously, in our implementation the results were first intermediately combined to save on ancilla qubits, allowing for the simulation of larger grid sizes.

In summary, the WFC problem is formulated as a Boolean search over tile configurations, with local adjacency constraints encoded into a reversible oracle using batched constraint evaluation and ancilla reuse. Grover's algorithm is then applied to amplify valid configurations.

Results

The central research question of this project was whether currently available, student-accessible quantum computing platforms are sufficiently mature to meaningfully implement Grover's algorithm for nontrivial problems such as games and puzzle-like search tasks. Across all experiments, a clear distinction emerges between theoretical performance, simulation results, and execution on real quantum hardware.

In ideal, noise-free simulations, Grover's algorithm consistently behaves as expected: valid solutions are amplified according to theoretical predictions, and the correct configurations dominate the measurement outcomes near the optimal number of iterations. These results confirm that the problem encodings, oracle constructions, and diffusion operators are logically correct and that Grover's algorithm can, in principle, be applied to structured problems beyond textbook examples.

When moving from simulation to realistic noise models and real quantum hardware, performance degrades significantly. While small instances occasionally retain partial amplification, increasing circuit depth and the number of multi-qubit gates quickly leads to decoherence and loss of signal. In practice, circuit depth, rather than raw qubit count, emerges as the dominant limiting factor. Compilation overhead, connectivity constraints, and accumulated gate errors often eliminate the theoretical advantage predicted by Grover's algorithm.

These results indicate that, although Grover's algorithm is theoretically applicable to game-like problems, its practical usefulness on current student-accessible quantum hardware is limited to very small problem instances. Larger instances remain feasible only in simulation, where hardware noise and coherence constraints are absent.

The following sections report the results separately for each studied problem.

Sudoku: The solver was first executed on a 2×2 Sudoku instance using IBM quantum hardware. Due to the small problem size, the resulting circuit depth was well within the coherence time of the processor. As shown in Figure 2, the hardware successfully identified the correct solutions. While it is clearly visible that noise is present, and it is manifesting as non-zero probabilities for invalid states, the bitstrings of the correct solutions stand out with a statistically significant probability peak.

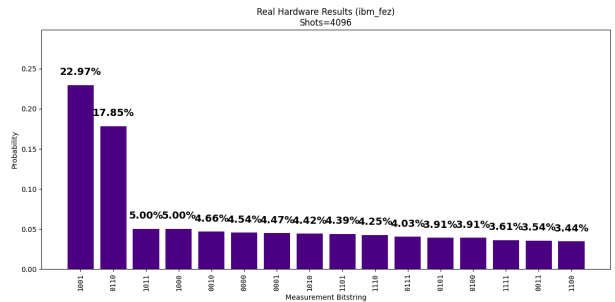


Figure 2: Hardware results for the 2×2 Sudoku instance. Unlike the larger case, the circuit depth here is shallow enough that the signal survives decoherence. Both highest bars represent the valid solutions, and are clearly distinguishable from the noise.

Following the successful proof of concept, the Grover search for the partially filled 4×4 Sudoku (6 unknowns) was executed on both a noiseless simulator and the `ibm_torino` quantum processor. The logical circuit required approximately 28,000 quantum gates and was set to run for 50 iterations, the theoretically optimal number calculated for this problem size.

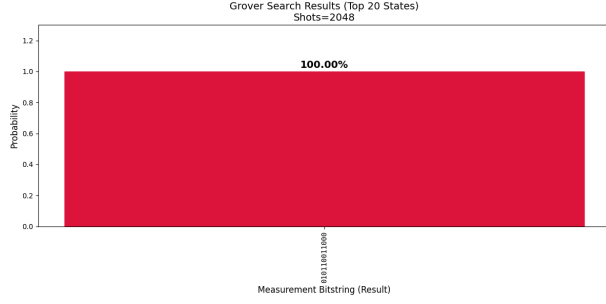


Figure 3: Ideal simulation results (50 Iterations). In the absence of noise, Grover’s algorithm converges to the correct state with unity probability, serving as the theoretical baseline for hardware comparisons.

The algorithm was run on the Qiskit Aer Statevector simulator. Figure 3 demonstrates the algorithm’s logical correctness: the probability of measuring the correct solution state (010110011000) reached 100%, confirming that the Oracle and Diffuser were constructed correctly and capable of amplifying the solution in a noise-free environment. Moreover, using a less than theoretically optimal number of iterations shows the expected Grover amplification pattern [2] (e.g., 97.61% at 45 iterations, see Figure 10, 51.61% at 25 iterations, see Figure 11, and $\sim 0.0035\%$ at only one iteration, see Figure 12), thus validating the theoretical model.

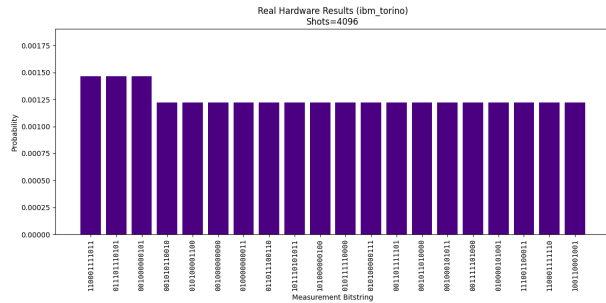


Figure 4: Experimental results on `ibm_torino` (4096 shots). The distribution is flat, indicating that the quantum state decohered into random noise before the computation could complete.

When the circuit was executed on the `ibm_torino` backend (133 qubits), the results were drastically different. As shown in Figure 4, the probability distribution is indistinguishable from uniform random noise ($\sim 0.015\%$). The amplification observed in the simulator was completely lost.

To understand the discrepancies between the simulation and the hardware execution, the circuit compilation and the probability of the algorithm to find the correct answer were analyzed in both theoretical and real scenarios.

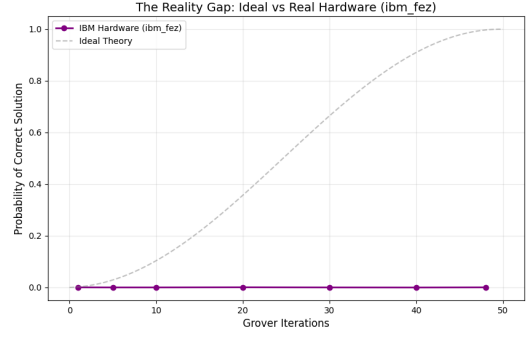


Figure 5: Coherence limit analysis. The ideal probability (grey) rises to 100%, while the real hardware probability (purple) never deviates from zero, showing that noise dominates the signal instantly.

A depth-sweep experiment was performed, running the algorithm for $k \in \{1, 5, \dots, 50\}$ iterations on hardware. Figure 5 illustrates the gap between the simulation and the real capabilities of a current quantum computer. While theory predicts a smooth sinusoidal rise in probability (grey line), the hardware success rate (purple line) remains flat at zero even for shallow depths. This indicates that the circuit depth exceeded the qubit coherence time (T_1/T_2) almost immediately.

The primary cause of this failure was identified in the transpilation phase of the process. The logical circuit contained approximately 28,100 gates, and to map this to the physical hardware, the insertion of SWAP gates was required to satisfy connectivity constraints. This caused a massive increase in the gate count to 785,780 and the circuit depth to 290,053. Given that current superconducting qubits can typically sustain only a few thousand gates before decoherence, a depth of 290,000 makes successful computation physically impossible for the currently available hardware.

Lights Out: The Lights Out Grover implementation was executed on three execution targets and the outcome distributions were compared: (1) a noiseless statevector simulator, (2) the Qiskit fake backend FakeMarrakesh (which includes simulated noise), and (3) real IBM hardware (`ibm_marrakesh`).

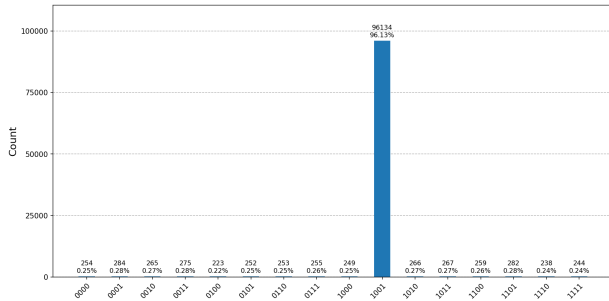
Key numerical results for the 2×2 instance (initial game state 1001, correct solution 1001):

- Perfect (statevector) simulator, 100k shots: measured solution frequency $\approx 96.13\%$; second-most frequent outcome was 1000 at $\approx 0.28\%$.
- FakeMarrakesh noise simulation, 10k shots: measured solution frequency $\approx 37.61\%$; second-most frequent outcome was 1011 at $\approx 6.03\%$.
- Real hardware (`ibm_marrakesh`), 100k shots: measured solution frequency $\approx 16.42\%$; second-most frequent outcome was 0001 at $\approx 6.90\%$ (significant degradation compared to the simulator).

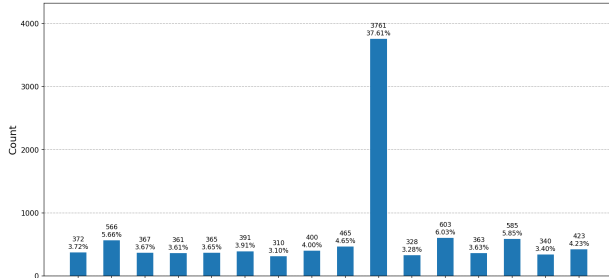
Before transpilation for IBM hardware, the circuit had 156 gates and the depth was 63. After transpilation for the target hardware, the circuit grew to 3287 gates with a depth of 971. The large increase in two-qubit operations (e.g., SWAP gates introduced due to connectivity constraints) dramatically increased circuit depth and size. This increase is the main contributor to the loss of signal on hardware.

Additionally, since the transpilation is non-deterministic, the amount of gates and the circuit’s depth are random. Therefore, a good transpilation seed was found to minimize the depth of the circuit.

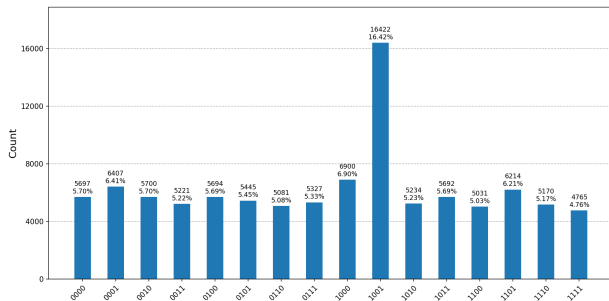
The simulator run confirms the correctness of the oracle and diffuser (see Figure 6). The noisy simulator shows that, under a realistic noise model, some amplification survives, whereas the hardware execution suffers from decoherence and compilation overhead.



(a) Perfect statevector simulator (100k shots)



(b) FakeMarrakesh (10k shots)



(c) IBM hardware (ibm_marrakesh, 100k shots)

Figure 6: Outcome histograms for the 2×2 Lights Out instance.

WFC: The Wave Function Collapse (WFC) implementation was evaluated on a 3×3 grid, which represents the largest instance that could be handled reliably within

the available simulation resources. Even at this size, the problem already exhibits a large number of valid configurations, making it a suitable benchmark to study Grover’s algorithm in a highly degenerate solution space. A quantitative summary of the experiment is therefore given first to establish the computational scale and observed performance.

Table 1 summarizes the quantum resource requirements and simulation results for the 3×3 instance. The encoding required 18 data qubits to represent tile values and 8 ancilla qubits for reversible constraint checking, resulting in a total of 26 qubits. The logical circuit depth before transpilation was 12, which increased to 2063 after transpilation for the simulator backend. In the noiseless Qiskit simulator, 1020 out of 1024 measurement shots corresponded to valid configurations, yielding a success probability of 99.6%. In total, 790 unique valid configurations were observed. Example valid tile configurations generated by the quantum algorithm are provided in Appendix A.

Table 1: Quantum Wave Function Collapse results for the 3×3 grid.

Metric	Value
Grid size	3×3
Qubits (data / ancilla / total)	18/8/26
Grover iterations	9
Circuit depth (pre / post transp.)	12/2063
Valid outcomes	1020/1024 (99.6%)

To study the behavior of Grover’s algorithm for this problem, the success probability was evaluated as a function of the number of Grover iterations. Figure 7 shows that, for the iteration range $k = 0$ to $k = 10$, the success probability follows a smooth rising curve that is consistent with a segment of the expected quadratic sinusoidal behavior. For larger iteration counts, the probability is expected to oscillate according to an overall quadratic sinusoidal envelope, as predicted by Grover theory. This can be seen for the 2×2 grid size shown in Appendix A. The relatively smooth behavior observed here is explained by the large fraction of marked states, which reduces the sharpness of the oscillations compared to problems with a unique solution.

The outcome distributions obtained from simulation and real quantum hardware are compared in Figure 8. In the noiseless Qiskit simulator, the top measured outcomes correspond exclusively to valid configurations, each with a probability on the order of 3.5×10^{-3} to 4.0×10^{-3} , indicating correct oracle and diffuser behavior. In contrast, execution on IBM quantum hardware shows substantial probability mass distributed over invalid configurations. Out of 1024 shots on hardware, only 14 valid configurations were observed, corresponding to a success probability of approximately 1.4%. The reason behind this decrease in probability can be seen when looking at the circuit depth. The circuit depth for the simulator case



Figure 7: Measured success probability as a function of the number of Grover iterations for the 3×3 WFC instance.

was around 2063, while the IBM hardware increases this to 169580. This is due to the compression of the multi-controlled X and Z gates into elementary gates.

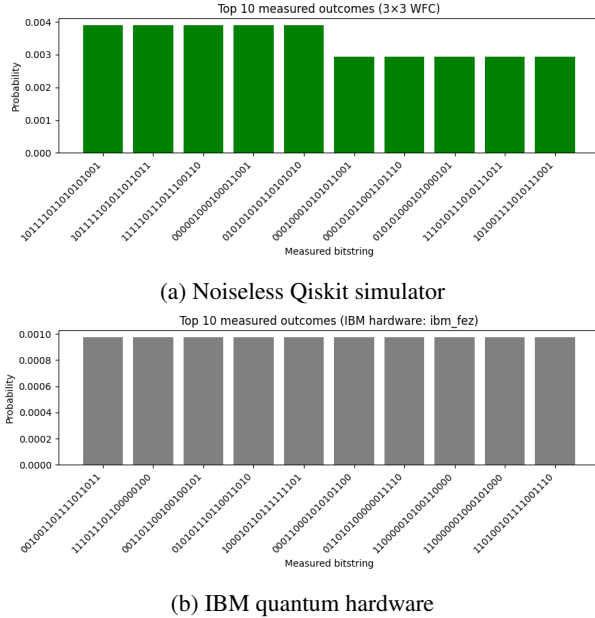


Figure 8: Measurement histograms for the 3×3 WFC instance. Green bars indicate valid configurations, while gray bars indicate invalid configurations. The simulator amplifies only valid solutions, whereas the hardware distribution is strongly degraded by noise and circuit depth.

Finally, the quantum WFC results are compared to a classical WFC implementation. Figure 9 shows an example map generated classically. Classical WFC produces valid 3×3 maps practically instantly. Our implementation has been optimized for our choice of ruleset, and has a runtime complexity of $O(n^4)$ for n by n grids, but for more complex rulesets the complexity may be up to $O(n^5)$. The runtime complexity of the quantum version of WFC is hard to analyze, because the number of valid results is very difficult to calculate for any arbitrary ruleset, which means the exact optimal number of iterations is unknown. In practice, however, the quantum implementation is greatly lim-

ited by the depth of the quantum circuit, as too deep circuits cause noise in the measurement result to the point of the result being purely random. We were able to simulate our quantum implementation for grid sizes up to 3 by 3, but all attempts to run the algorithm on actual hardware unfortunately failed. This means that for any grid size, the classical implementation remains superior for now.



Figure 9: Example of a tile map generated using our classical Wave Function Collapse solver implementation. Here, Water tiles are in blue, Beach tiles in yellow, Grass tiles in light green and Forest tiles in dark green.

Discussion

The experiments show a clear gap between the *theoretical* speed-up of Grover’s algorithm and its *practical* usefulness on student-accessible quantum hardware. In noise-free simulations, Grover amplification behaves as expected for all three problems, confirming that the encodings, oracles, and diffusion operators are logically correct. On real hardware, however, performance is dominated by circuit depth and two-qubit gate errors, so the theoretical advantage is usually lost before a useful number of Grover iterations can be executed.

A main reason Grover underperforms for small instances is constant overhead. Turning a classical constraint check into a reversible oracle requires extra ancilla qubits, multi-controlled operations, and uncomputation. For the instance sizes that fit on hardware, this overhead can be comparable to (or exceed) the full runtime of classical solvers. In addition, all three problems have an exploitable structure. Sudoku can be solved efficiently by classical backtracking with pruning, and Lights Out can be solved via linear algebra. WFC uses local constraint propagation and collapse, which produces valid maps quickly in practice. Grover’s quadratic improvement applies to *unstructured brute-force search*, but the strongest classical baselines are *structure-aware*, which is why

classical methods dominate for the tested instance sizes.

Across all three games, the most important practical limiter is circuit depth after transpilation. Limited device connectivity introduces SWAP gates and increases two-qubit gate volume, which rapidly accumulates error. This effect is visible in Lights Out, where the circuit grows from 156 logical gates to 3287 after transpilation, and the measured solution frequency drops from $\approx 96.13\%$ (noiseless simulator) to $\approx 37.61\%$ (noise model) and $\approx 16.42\%$ (hardware). In Sudoku, the 2×2 instance still demonstrates Grover successfully on hardware, but the partially filled 4×4 case fails despite sufficient qubits because the oracle is deep and requires many Grover iterations; compilation overhead and noise flatten the distribution. In WFC, the 3×3 simulator run achieves 99.6% valid outputs (1020/1024), yet the IBM hardware run yields only $\approx 1.4\%$ valid outputs (14/1024), consistent with the transpiled depth rising to $\mathcal{O}(10^5)$ and overwhelming any amplification.

Ancilla qubits create a trade-off that affects all three implementations. More ancillae can reduce depth by enabling more efficient decompositions of multi-controlled gates, but they also increase routing complexity and can increase SWAP overhead. Therefore, reducing qubits can sometimes help (less routing) and sometimes hurt (deeper decompositions and more sequential operations). In practice, the best choice depends on the target backend and circuit layout, not only on the algorithmic design.

Looking forward, improvements in (i) two-qubit error rates, (ii) coherence times, and (iii) connectivity would directly increase the maximum usable circuit depth and allow more Grover iterations before the signal is lost. Error correction would change the picture more fundamentally, but it requires large overhead beyond current student-accessible devices. On the algorithm side, progress would come from shallower oracles (better constraint encodings, improved MCX synthesis, and ancilla reuse) and hybrid approaches where classical pruning reduces the search space before applying Grover. Even with better hardware, highly structured problems are likely to remain classical-favorable unless the quantum approach also exploits structure rather than only accelerating brute-force search.

Conclusion

This project evaluated whether student-accessible quantum computing platforms are currently well enough to implement Grover’s algorithm meaningfully on game-like problems, and at what scale this is feasible. The results show that Grover is currently most useful as a proof-of-concept: small instances can demonstrate correct behavior, but scalability is limited by circuit depth and noise.

For Sudoku, Grover works well for a 2×2 instance

on real hardware, but the partially filled 4×4 case remains practical only in simulation and is outperformed by classical structure-aware solvers. For Lights Out, correct amplification is observed in simulation, yet hardware performance degrades strongly due to transpilation overhead and two-qubit gate noise. For WFC, the 3×3 case achieves high success probability in simulation (99.6%), while hardware results are near-uniform with only $\approx 1.4\%$ valid outcomes, reflecting circuit depths far beyond coherence limits.

Overall, Grover helps when the oracle is compact and the instance is small enough that depth stays within hardware limits. It does not yet provide a practical advantage over classical methods for these structured problems on current student-accessible hardware. Future work should focus on reducing oracle depth, exploring hybrid classical–quantum pipelines (e.g., classical pruning followed by Grover), and repeating these experiments as hardware improves, especially with better connectivity and lower two-qubit error rates.

Author Contributions

- **H. Botezatu:** Implemented and analyzed the Sudoku experiments with a strong focus on the 4x4 case; contributed to writing the Sudoku methodology and results.
- **R. Kapoor:** Contributed to the quantum WFC methodology and implementation, and contributed to writing the abstract, discussion, and conclusion.
- **L. Milieška:** Implemented and analyzed the Lights Out experiments; contributed to writing the Lights Out methodology and results.
- **F. van Onna:** Wrote the classical WFC solver implementation, contributed to the quantum WFC methodology and implementation.
- **D. Lenta:** Implemented and analyzed the Lights Out classical solver; contributed to writing the Lights Out methodology and results.
- **R. van Unen:** Implemented and analyzed the Sudoku experiments; contributed to writing the Sudoku, methodology, introduction, abstract and results.

References

- [1] Lov K. Grover. “A Fast Quantum Mechanical Algorithm for Database Search”. In: *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing (STOC '96)*. New York, NY, USA: Association for Computing Machinery, 1996, pp. 212–219. DOI: 10.1145/237814.237866. URL: <https://doi.org/10.1145/237814.237866>.
- [2] Stefano Bosco. *Quantum Computation and Communication – Lecture 5*. Lecture slides. 2024.
- [3] URL: https://quantum.cloud.ibm.com/docs/en/api/qiskit/synthesis#synth_mcx_n_clean_m15.
- [4] Tanuj Khattar and Craig Gidney. “Rise of conditionally clean ancillae for efficient quantum circuit constructions”. In: *Quantum* 9 (May 2025), p. 1752. ISSN: 2521-327X. DOI: 10.22331/q-2025-05-21-1752. URL: <http://dx.doi.org/10.22331/q-2025-05-21-1752>.
- [5] OEIS Foundation Inc. (2026), Entry A075462 in *The On-Line Encyclopedia of Integer Sequences*, <https://oeis.org/A075462>.
- [6] URL: <https://quantum.cloud.ibm.com/docs/en/guides/dynamical-decoupling-pass-manager>.

Appendix A

This section includes additional experimental results.

Sudoku: For the Sudoku experiments, this section presents additional results from different measurements.

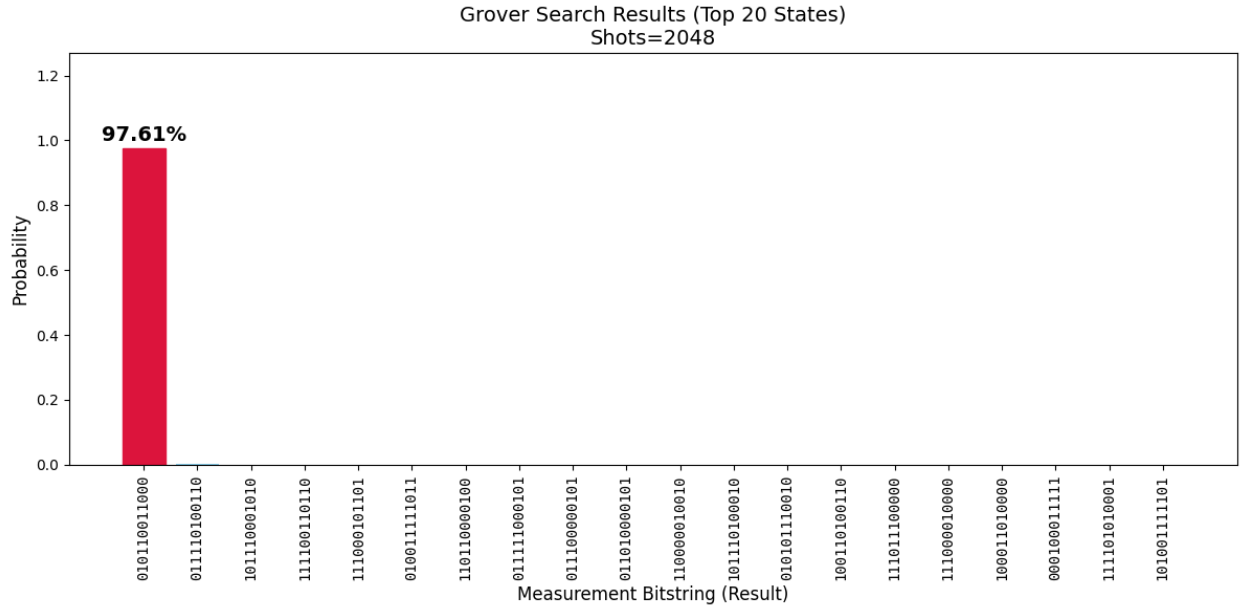


Figure 10: Simulation results for 45 iterations showing 97.61% probability.

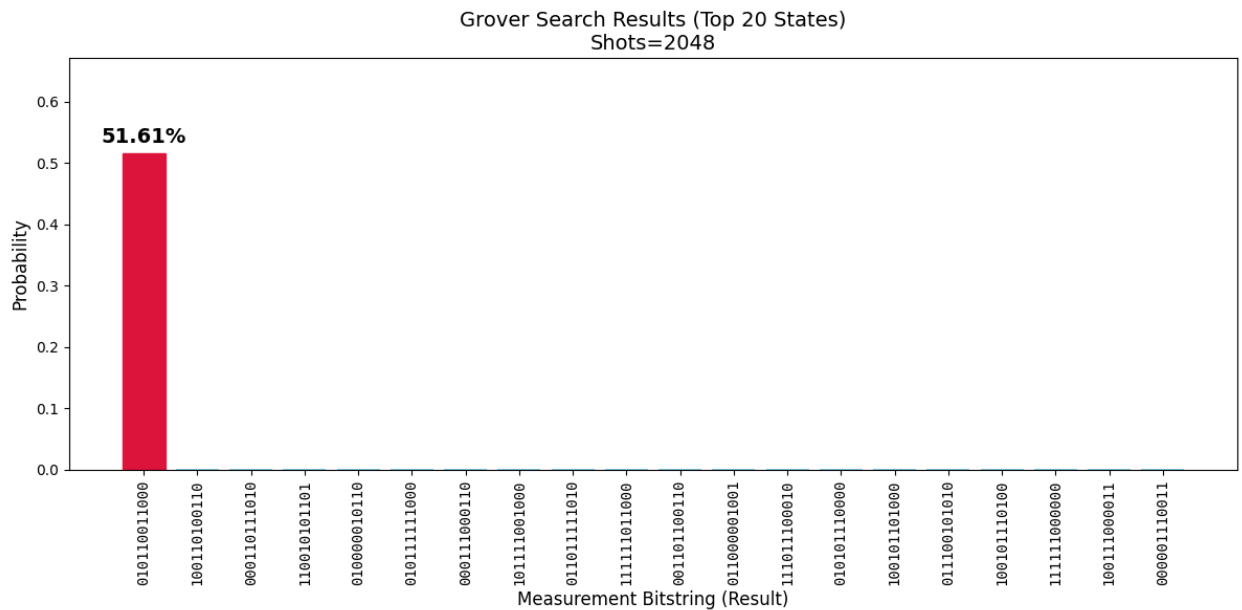


Figure 11: Simulation results for 25 iterations showing 51.61% probability.

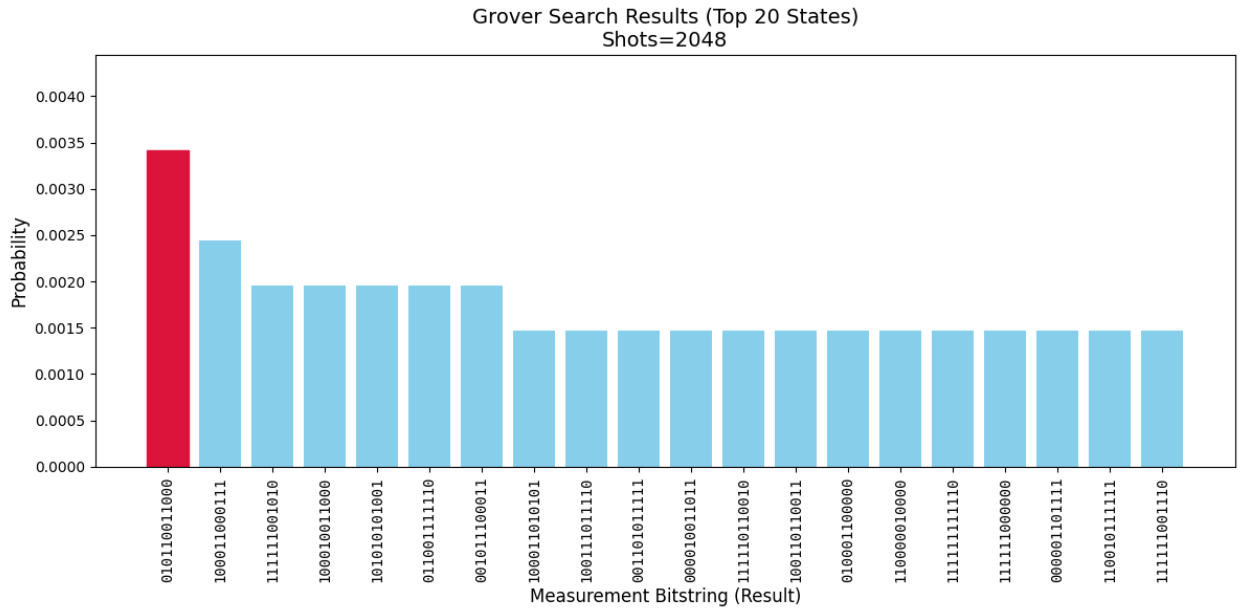


Figure 12: Simulation results for one iteration showing $\sim 0.0035\%$ probability.

WFC: For the Wave Function Collapse method, this section presents additional map generations and graphs for different grid sizes.

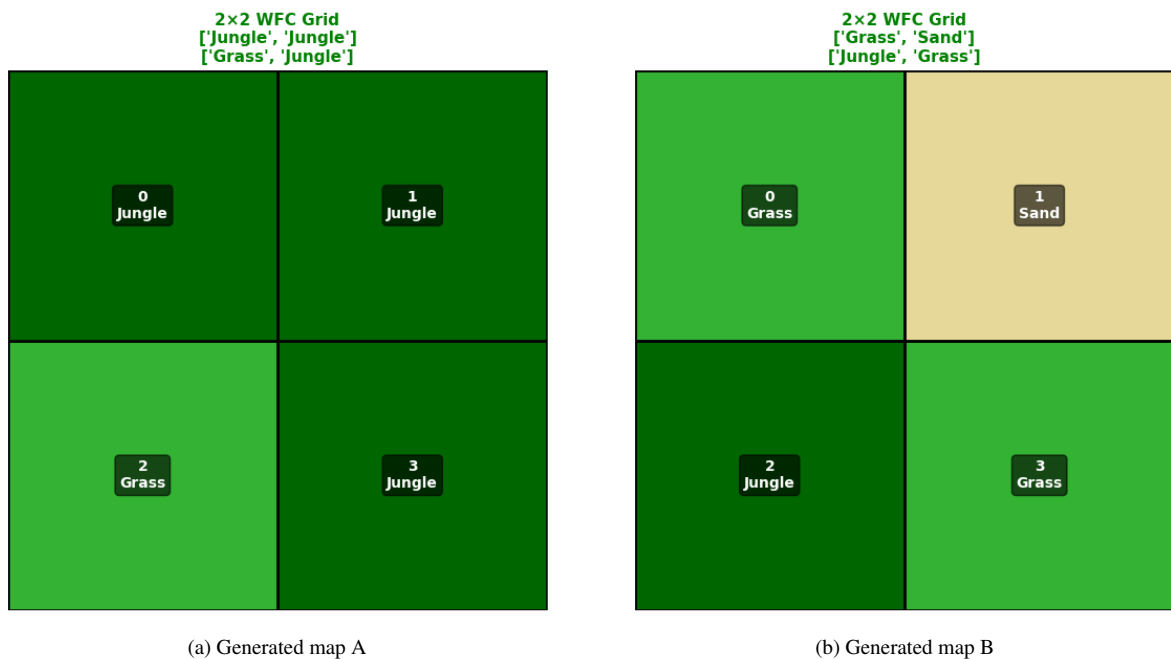


Figure 13: 2x2 maps generated using the quantum Wave Function Collapse (WFC) method with Aer.

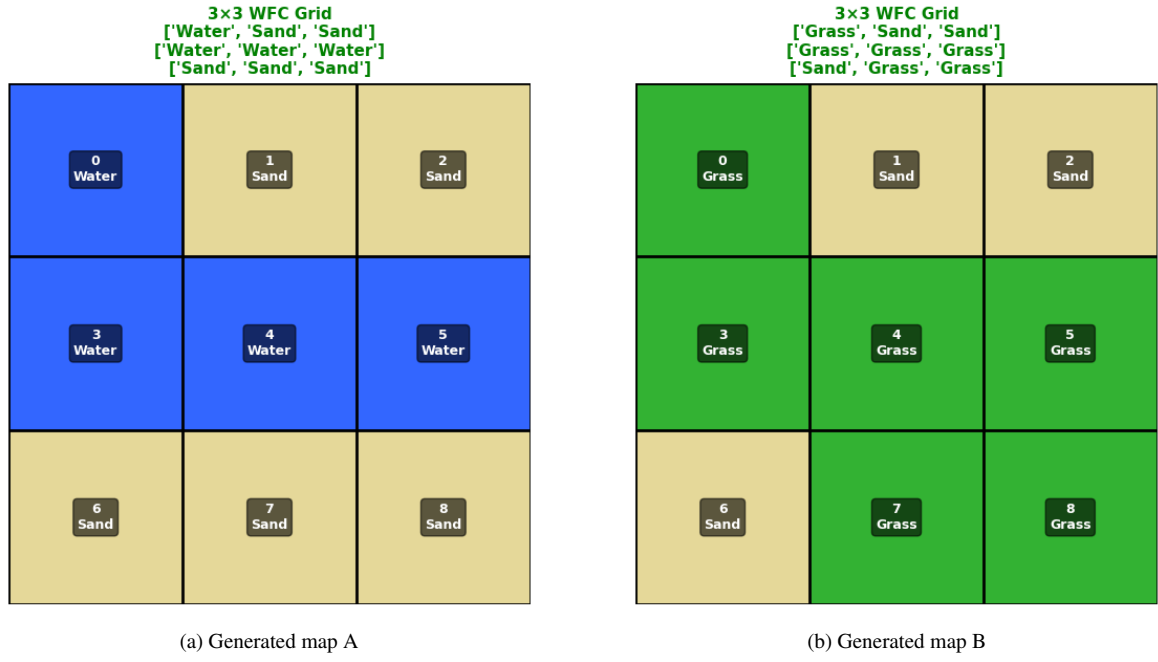


Figure 14: 3×3 maps generated using the quantum Wave Function Collapse (WFC) method with Aer.

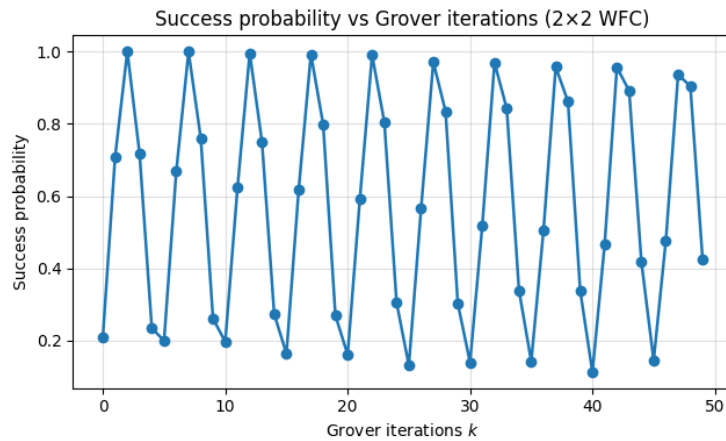


Figure 15: Probability of obtaining a valid 2×2 map as a function of the number of Grover iterations. The success probability follows the characteristic quadratic sinusoidal behavior of Grover's algorithm.

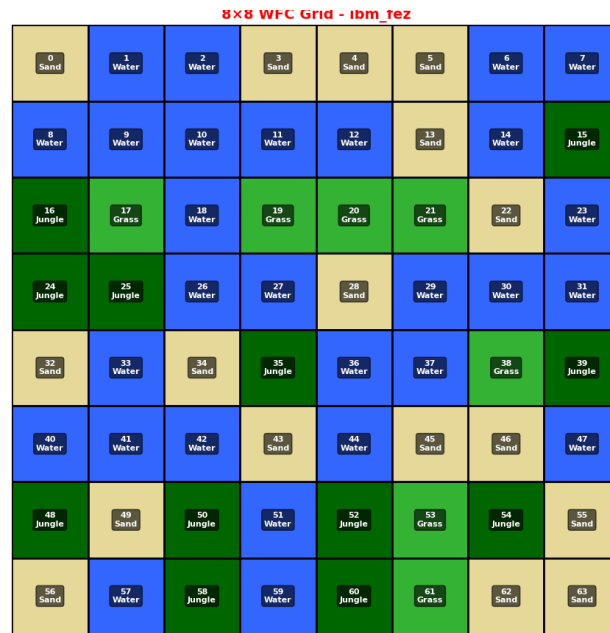


Figure 16: 8×8 map (largest possible map to generate) using the quantum Wave Function Collapse (WFC) method with IBM hardware.

Appendix B

This GitHub repository contains the code used during the project: <https://github.com/Rudieeeee/QuantumMinorProject>